Kaushik Narayanan V

192321047

CSA1735 – Artificial Intelligence

QUESTION 11: MAP COLORING USING CSP

AIM: To implement map coloring as a Constraint Satisfaction Problem (CSP), assigning colors to regions so that neighboring regions have different colors.

ALGORITHM:

1. 1. Define the regions and their neighboring relationships.
2. 2. Define the available colors and create an empty assignment.
3. 3. Select the next uncolored region.
4. 4. Try each color and check whether it conflicts with any already-colored neighbor.
5. 5. If consistent, assign the color and recursively solve the remaining regions.
6. 6. If no color works, backtrack and try another color.
7. 7. Print the assignment if a complete solution is found.

PROGRAM:

Kaushik Narayanan V - 192321047

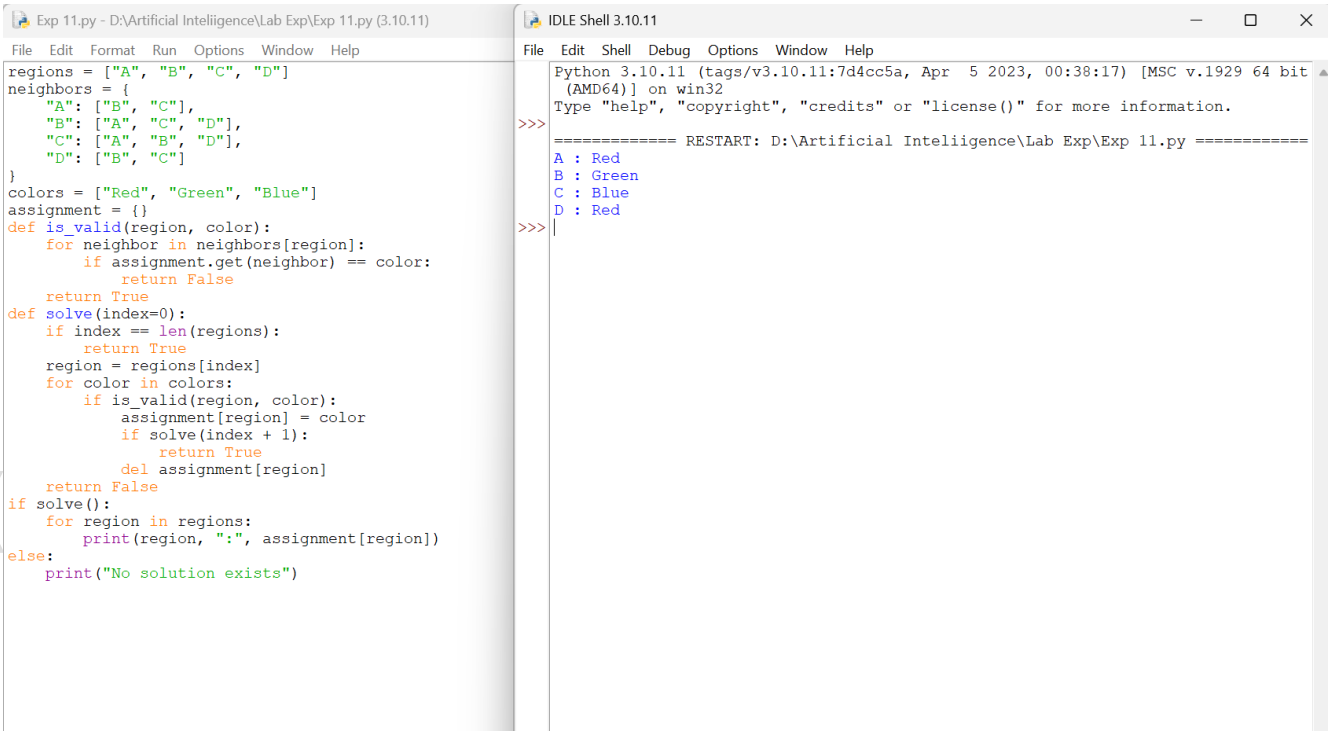
```
# Map coloring using Constraint Satisfaction Problem (CSP)
regions = ["A", "B", "C", "D"]
neighbors = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}
colors = ["Red", "Green", "Blue"]
assignment = {}

def is_consistent(region, color):
    for neighbor in neighbors[region]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve(index=0):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_consistent(region, color):
            assignment[region] = color
            if solve(index + 1):
```

```
            return True
        del assignment[region]
    return False

if solve():
    print("Map coloring solution:")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution exists.")
```



RESULT: The map-coloring CSP was implemented successfully using backtracking, and adjacent regions receive different colors.

QUESTION 12: TIC-TAC-TOE GAME

AIM: To develop a two-player Tic-Tac-Toe game in Python in which players take turns marking a 3 × 3 board and the program detects a win or draw.

ALGORITHM:

1. Initialize a board containing nine empty cells.
2. Display the board in a 3 × 3 format.
3. Ask the current player to enter a position from 1 to 9.
4. Validate the position and reject invalid or occupied cells.
5. Place the player's mark and check all winning rows, columns, and diagonals.
6. If a player wins, display the winner and stop the game.
7. Otherwise switch players; after nine valid moves without a winner, declare a draw.

PROGRAM:

```
# Tic-Tac-Toe game for two human players
board = [" "] * 9

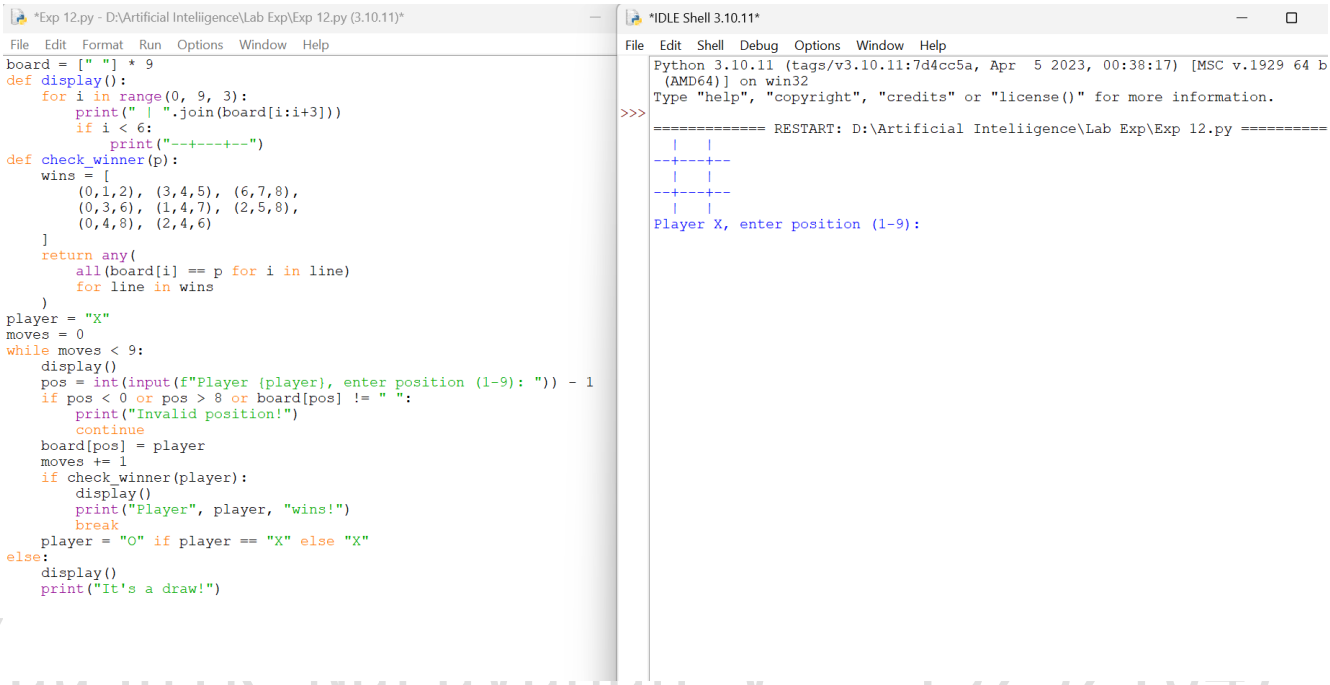
def display_board():
    print()
    for row in range(3):
        start = row * 3
        print(" " + " | ".join(board[start:start + 3]))
        if row < 2:
            print("---+---+---")

def winner(mark):
    lines = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    return any(all(board[i] == mark for i in line) for line in lines)

player = "X"
for turn in range(9):
    display_board()
    try:
        position = int(input(f"Player {player}, choose position (1-9): ")) - 1
    except ValueError:
        print("Enter a number from 1 to 9.")
        continue
    if position < 0 or position > 8 or board[position] != " ":
        print("Invalid or occupied position. Try again.")
        continue
    board[position] = player
    if winner(player):
        display_board()
```

Kaushik Narayanan V - 192321047

```
        print(f"Player {player} wins!")
        break
    player = "O" if player == "X" else "X"
else:
    display_board()
    print("It's a draw!")
```



Exp 12.py - D:\Artificial Inteliigence\Lab Exp\Exp 12.py (3.10.11)

File Edit Format Run Options Window Help

```
board = [" "] * 9
def display():
    for i in range(0, 9, 3):
        print(" | ".join(board[i:i+3]))
        if i < 6:
            print("--+---+--")
def check_winner(p):
    wins = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    return any(
        all(board[i] == p for i in line)
        for line in wins
    )
player = "X"
moves = 0
while moves < 9:
    display()
    pos = int(input(f"Player {player}, enter position (1-9): ")) - 1
    if pos < 0 or pos > 8 or board[pos] != " ":
        print("Invalid position!")
        continue
    board[pos] = player
    moves += 1
    if check_winner(player):
        display()
        print("Player", player, "wins!")
        break
    player = "O" if player == "X" else "X"
else:
    display()
    print("It's a draw!")
```

IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 b
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: D:\Artificial Inteliigence\Lab Exp\Exp 12.py ==========
  |   |
--+---+--
  |   |
--+---+--
  |   |
Player X, enter position (1-9):
```

RESULT: The Tic-Tac-Toe game was implemented with input validation, win detection, and draw detection.

QUESTION 13: MINIMAX ALGORITHM FOR GAMING

AIM: To implement the Minimax algorithm to select the best outcome for a maximizing player by assuming the opponent plays optimally.

ALGORITHM:

15. 1. Represent the game tree using nodes and child relationships.
16. 2. Assign utility values to terminal (leaf) nodes.
17. 3. At a leaf node, return its utility value.
18. 4. At a MAX node, return the maximum value among its children.
19. 5. At a MIN node, return the minimum value among its children.
20. 6. Evaluate the tree recursively and print the value selected for the root.

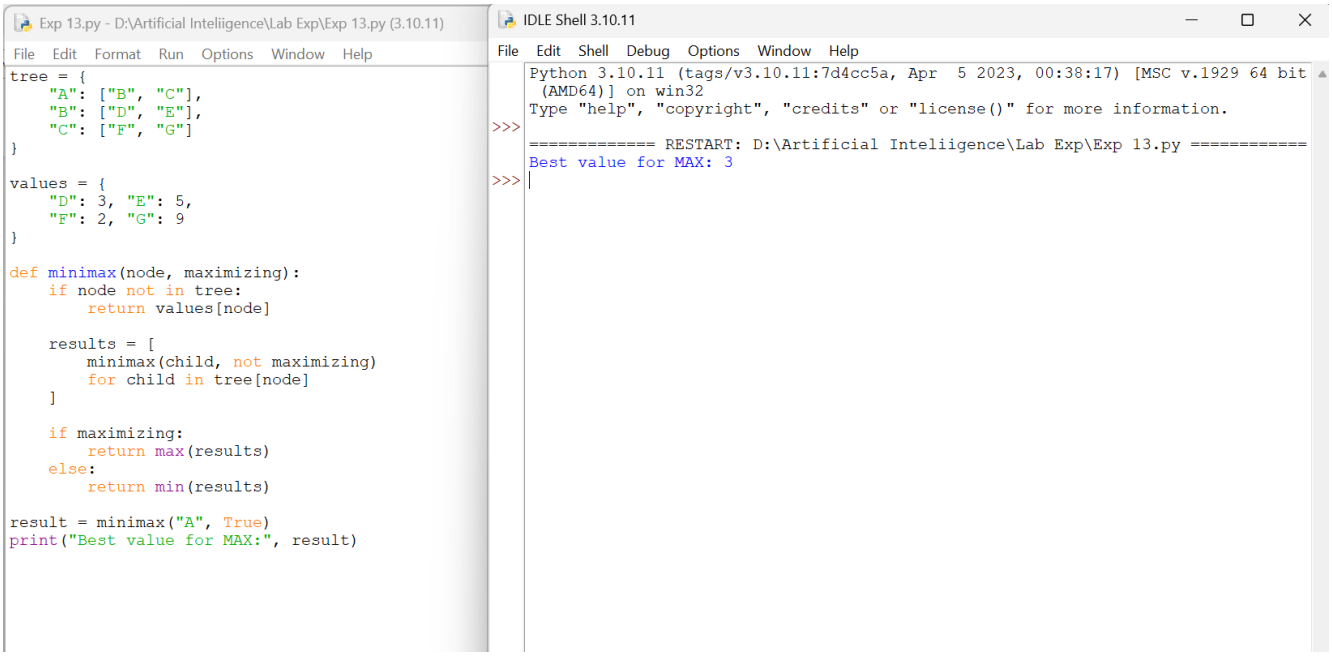
PROGRAM:

```
# Minimax algorithm on a small game tree
# Leaf values represent utility for MAX.
tree = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"]
}
values = {"D": 3, "E": 5, "F": 2, "G": 9}
```

Kaushik Narayanan V - 192321047

```
def minimax(node, maximizing):
    if node not in tree:
        return values[node]
    child_values = [minimax(child, not maximizing) for child in tree[node]]
    return max(child_values) if maximizing else min(child_values)

best_value = minimax("A", True)
print("Best value for MAX:", best_value)
```



RESULT: The Minimax algorithm evaluated the game tree and returned the best achievable value for MAX, assuming optimal play by both players.

Kaushik Narayanan V - 192321047

QUESTION 14: ALPHA-BETA PRUNING ALGORITHM

AIM: To implement Alpha-Beta pruning, an optimization of Minimax that skips branches that cannot affect the final decision.

ALGORITHM:

21. 1. Represent the game tree and assign values to leaf nodes.
22. 2. Initialize alpha to negative infinity and beta to positive infinity.
23. 3. At MAX nodes, update the best value and alpha.
24. 4. At MIN nodes, update the best value and beta.
25. 5. Stop exploring a node's remaining children when alpha is greater than or equal to beta.
26. 6. Return the selected root value and report any skipped branches.

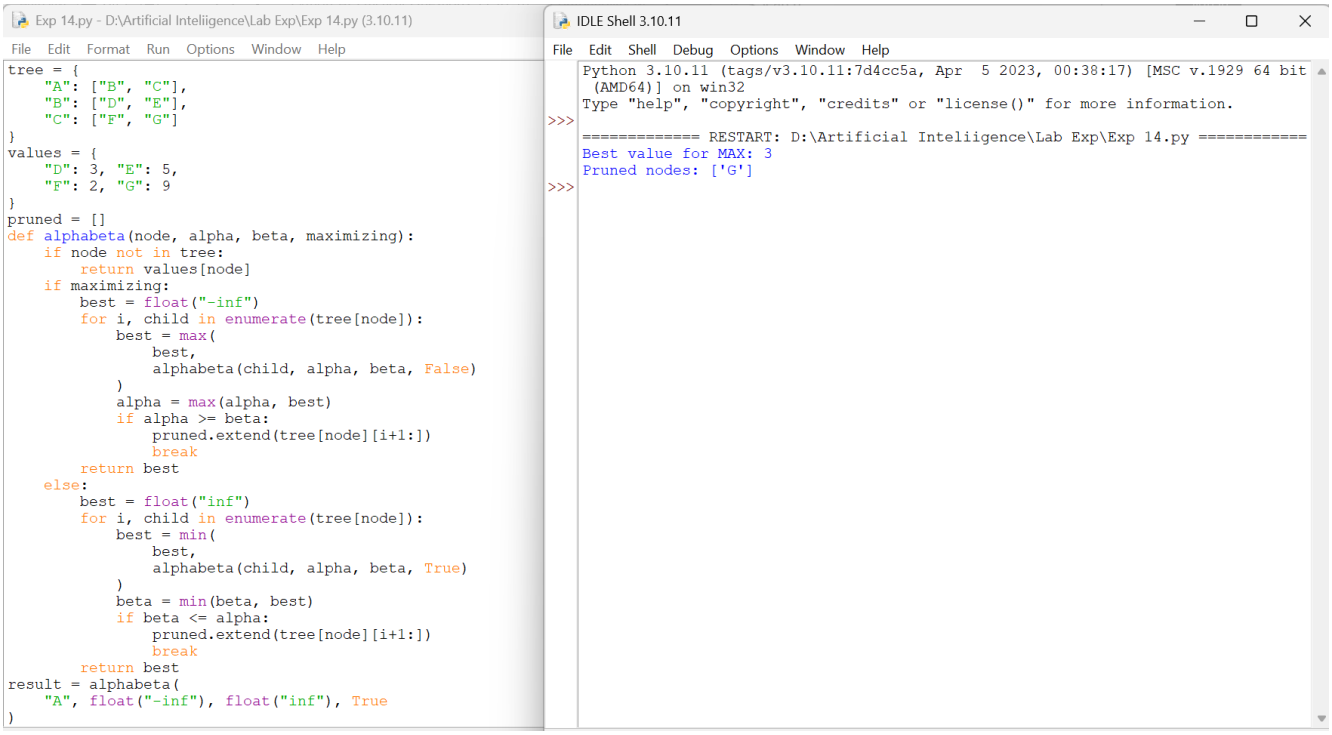
PROGRAM:

```
# Alpha-Beta pruning on a small game tree
tree = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"]
}
values = {"D": 3, "E": 5, "F": 2, "G": 9}
pruned = []
```

Kaushik Narayanan V - 192321047

```
def alphabeta(node, alpha, beta, maximizing):
    if node not in tree:
        return values[node]
    if maximizing:
        value = float("-inf")
        for child in tree[node]:
            value = max(value, alphabeta(child, alpha, beta, False))
            alpha = max(alpha, value)
            if alpha >= beta:
                pruned.extend(tree[node][tree[node].index(child) + 1:])
                break
        return value
    else:
        value = float("inf")
        for child in tree[node]:
            value = min(value, alphabeta(child, alpha, beta, True))
            beta = min(beta, value)
            if beta <= alpha:
                pruned.extend(tree[node][tree[node].index(child) + 1:])
                break
        return value

result = alphabeta("A", float("-inf"), float("inf"), True)
print("Best value for MAX:", result)
print("Pruned nodes:", pruned if pruned else "None")
```



RESULT: Alpha-Beta pruning returned the same optimal root value as Minimax while avoiding evaluation of a branch that cannot affect the decision.

Kaushik Narayanan V - 192321047

QUESTION 15: DECISION TREE

AIM: To train a Decision Tree classifier and predict whether a student passes or fails based on study hours and attendance percentage.

ALGORITHM:

27. 1. Import DecisionTreeClassifier from scikit-learn.
28. 2. Prepare a small dataset of study hours, attendance, and pass/fail labels.
29. 3. Create the Decision Tree model.
30. 4. Train the model using the feature data and labels.
31. 5. Read study hours and attendance from the user.
32. 6. Predict the class for the entered values and display Pass or Fail.

PROGRAM:

```
# Decision Tree classifier using scikit-learn
# Install dependency once: pip install scikit-learn
from sklearn.tree import DecisionTreeClassifier

# Features: [hours studied, attendance percentage]
X = [
    [1, 50], [2, 55], [3, 60], [4, 65],
    [5, 75], [6, 80], [7, 85], [8, 95]
]
# 0 = Fail, 1 = Pass
y = [0, 0, 0, 0, 1, 1, 1, 1]

model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

hours = float(input("Enter hours studied: "))
attendance = float(input("Enter attendance percentage: "))
prediction = model.predict([[hours, attendance]])[0]

print("Prediction:", "Pass" if prediction == 1 else "Fail")
```

Kaushik Narayanan V - 192321047

Exp 15.py - D:\Artificial Inteliigence\Lab Exp\Exp 15.py (3.10.11)

File Edit Format Run Options Window Help

```
# Decision Tree classifier without sklearn

X = [
    [1, 50], [2, 55], [3, 60], [4, 65],
    [5, 75], [6, 80], [7, 85], [8, 95]
]

y = [0, 0, 0, 0, 1, 1, 1, 1]

hours = float(input("Enter hours studied: "))
attendance = float(input("Enter attendance percentage: "))

# Decision tree rules
if hours <= 4:
    prediction = 0
else:
    prediction = 1

if prediction == 1:
    print("Prediction: Pass")
else:
    print("Prediction: Fail")
```

IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
============= RESTART: D:\Artificial Inteliigence\Lab Exp\Exp 15.py ============
Enter hours studied: 8
Enter attendance percentage: 80
Prediction: Pass
>>>
```

RESULT: The Decision Tree classifier was trained and used to predict a student's pass/fail category. The example dataset is illustrative.

Kaushik Narayanan V - 192321047

QUESTION 16: FEEDFORWARD NEURAL NETWORK

AIM: To implement a feedforward neural network that learns the XOR function and predicts an output for each binary input pair.

ALGORITHM:

33. 1. Import MLPClassifier from scikit-learn.
34. 2. Prepare the four XOR input combinations and their expected outputs.
35. 3. Create a multilayer perceptron with one hidden layer.
36. 4. Train the neural network on the XOR dataset.
37. 5. Pass each input combination through the trained network.
38. 6. Display the predicted output for each input.

PROGRAM:

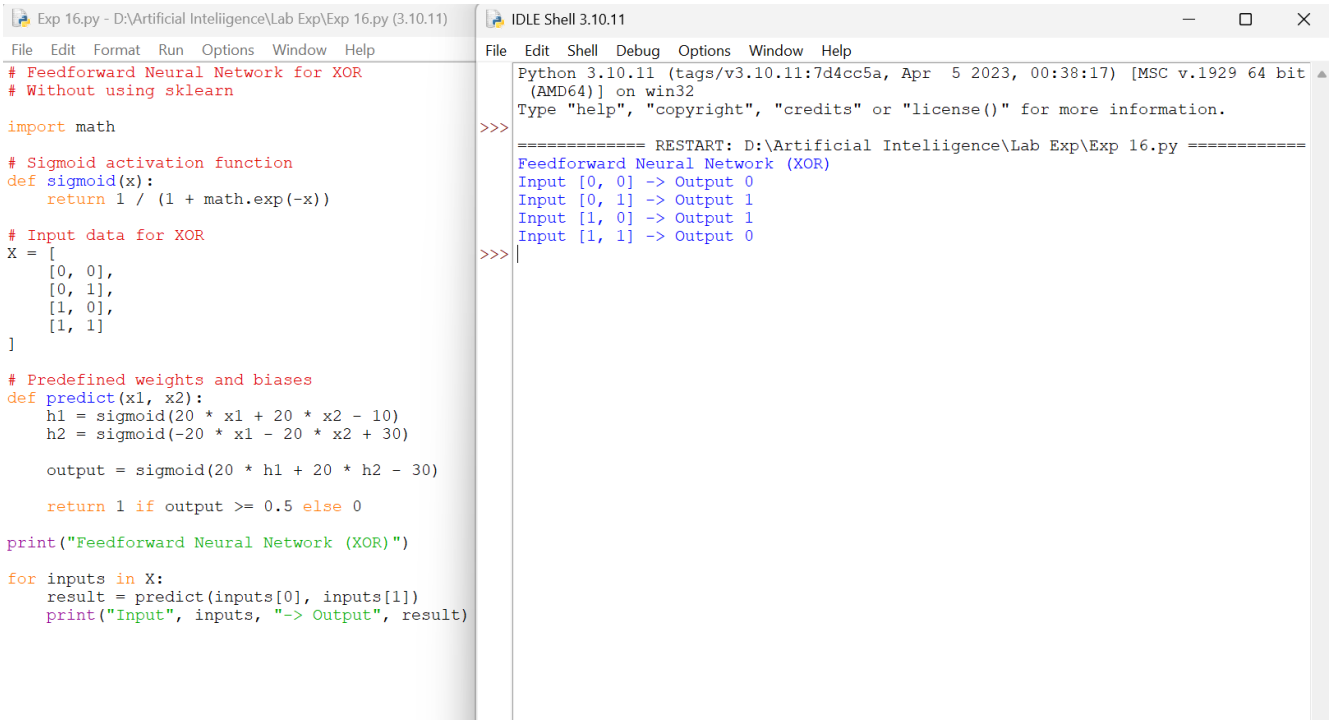
```
# Simple feedforward neural network using scikit-learn
# Install dependency once: pip install scikit-learn
from sklearn.neural_network import MLPClassifier

# XOR dataset
X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]

model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation="tanh",
    solver="lbfgs",
    max_iter=2000,
    random_state=1
)
model.fit(X, y)

print("Feedforward Neural Network (XOR)")
for sample in X:
    prediction = model.predict([sample])[0]
    print(f"Input {sample} -> Output {prediction}")
```

Kaushik Narayanan V - 192321047



RESULT: The feedforward neural network was trained on the XOR dataset and used to produce predictions for all four input combinations.

Kaushik Narayanan V - 192321047